ArchiHotel — Reservation Handling & Check-In

Enterprise Architecture of Invisible Luxury

Team C2F1G4 · Katrin Schwab · Soyeong Jeong · Fiona Aurelia

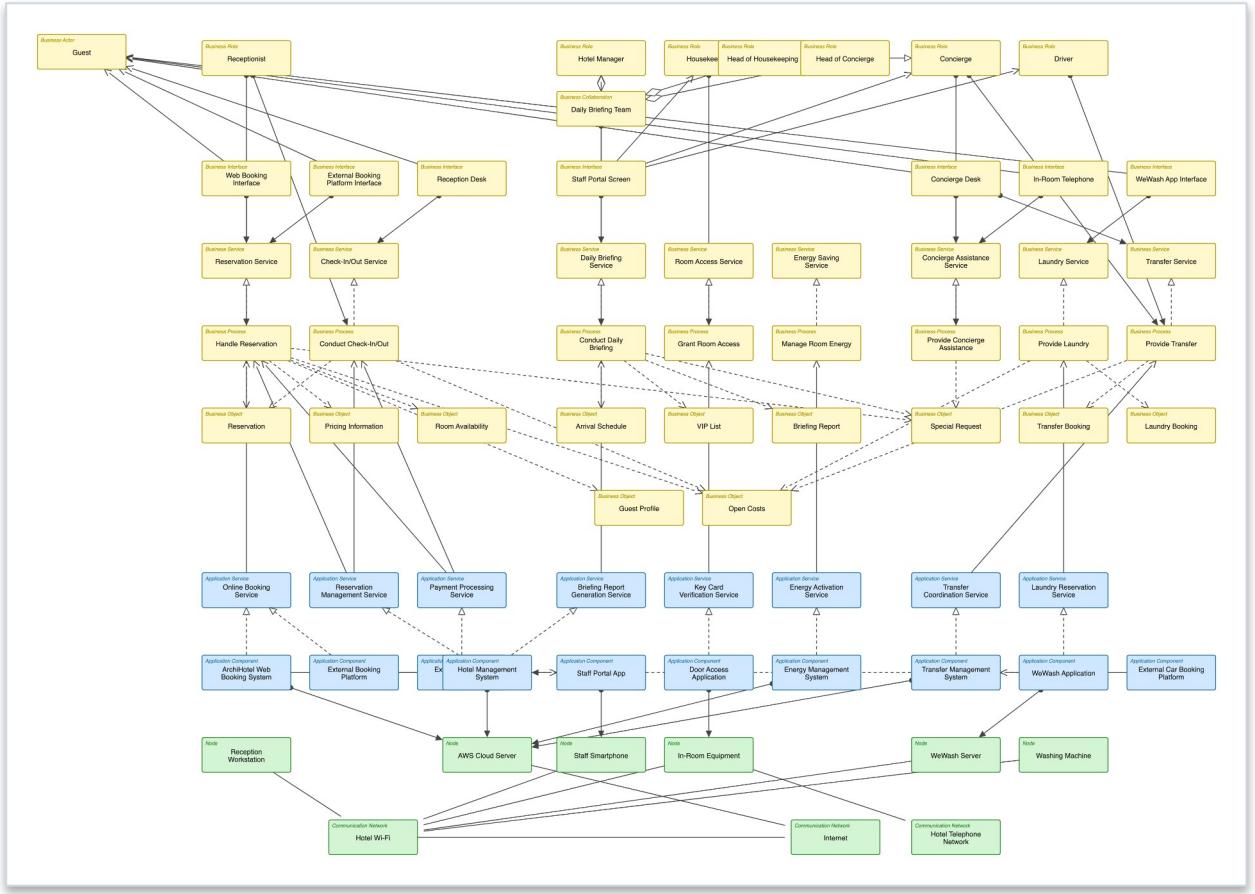
Technical University of Munich — School of Computation, Information and Technology
Information Systems · EAM and Reference Models — Case Study Final Presentation
Heilbronn · 06.07.2026

Introduction: The ArchiHotel Case

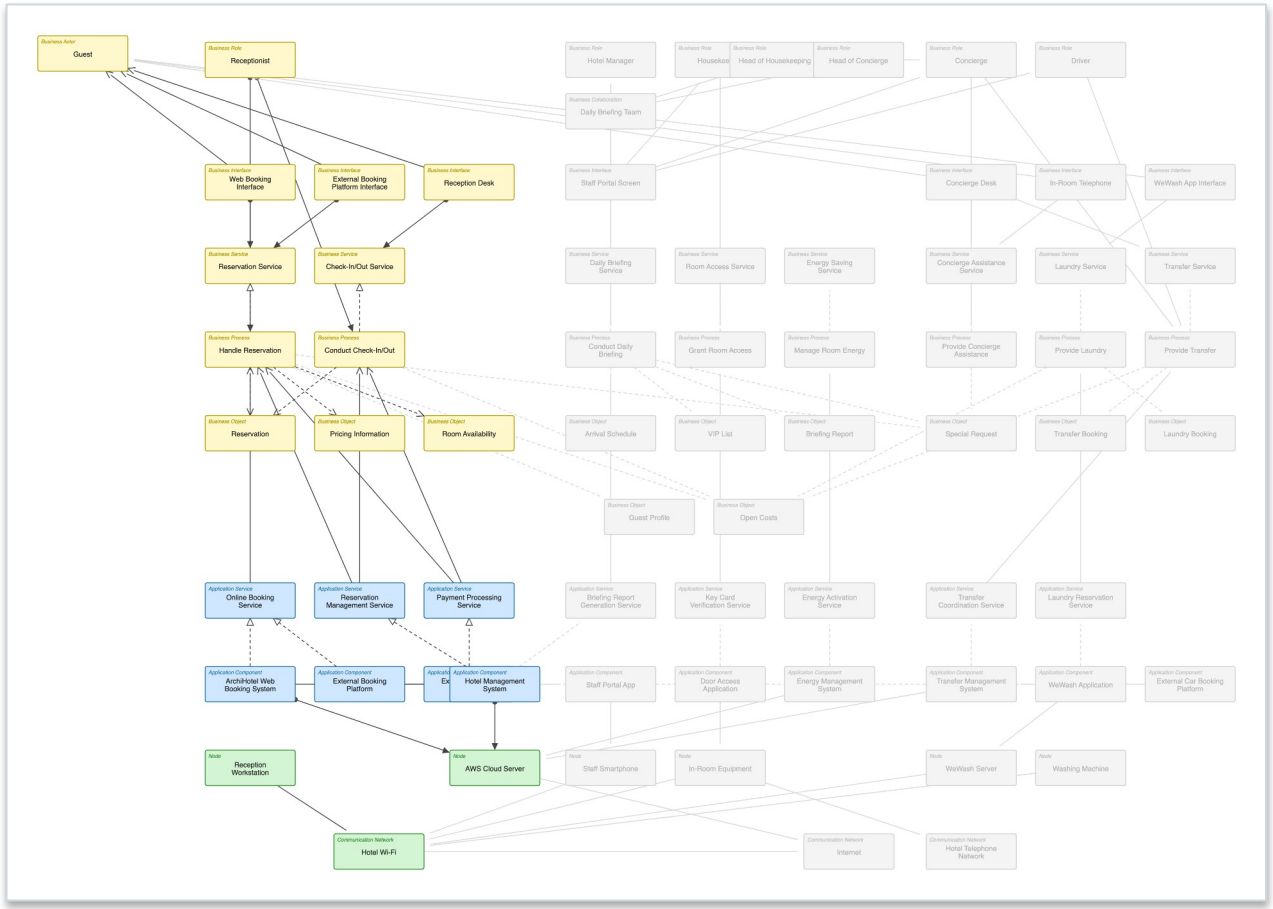
- ArchiHotel — a luxury hotel group: “where luxury feels effortless”
- People, systems and spaces are aligned before the guest ever notices
- Every department works from the same shared data:
  - Reception → Reservations, Open Costs
  - Housekeeping → Arrival Schedule, room status
  - Concierge → Transfers, VIPs, Special Requests
  - Operational leads → a complete view of the day
- Everything converges on the Hotel Management System (HMS)
- Our focus area — F1: Reservation Handling & Check-In

The Abstract EA Model

- The whole architecture at an abstract level
  - An overview — please don't read every box
- HMS sits at the structural centre
- AWS Cloud Server hosts the core systems
- Three networks: Hotel Wi-Fi, Internet, Telephone
- All three layers · 70 elements / 95 relationships
- → We now zoom into the F1 region



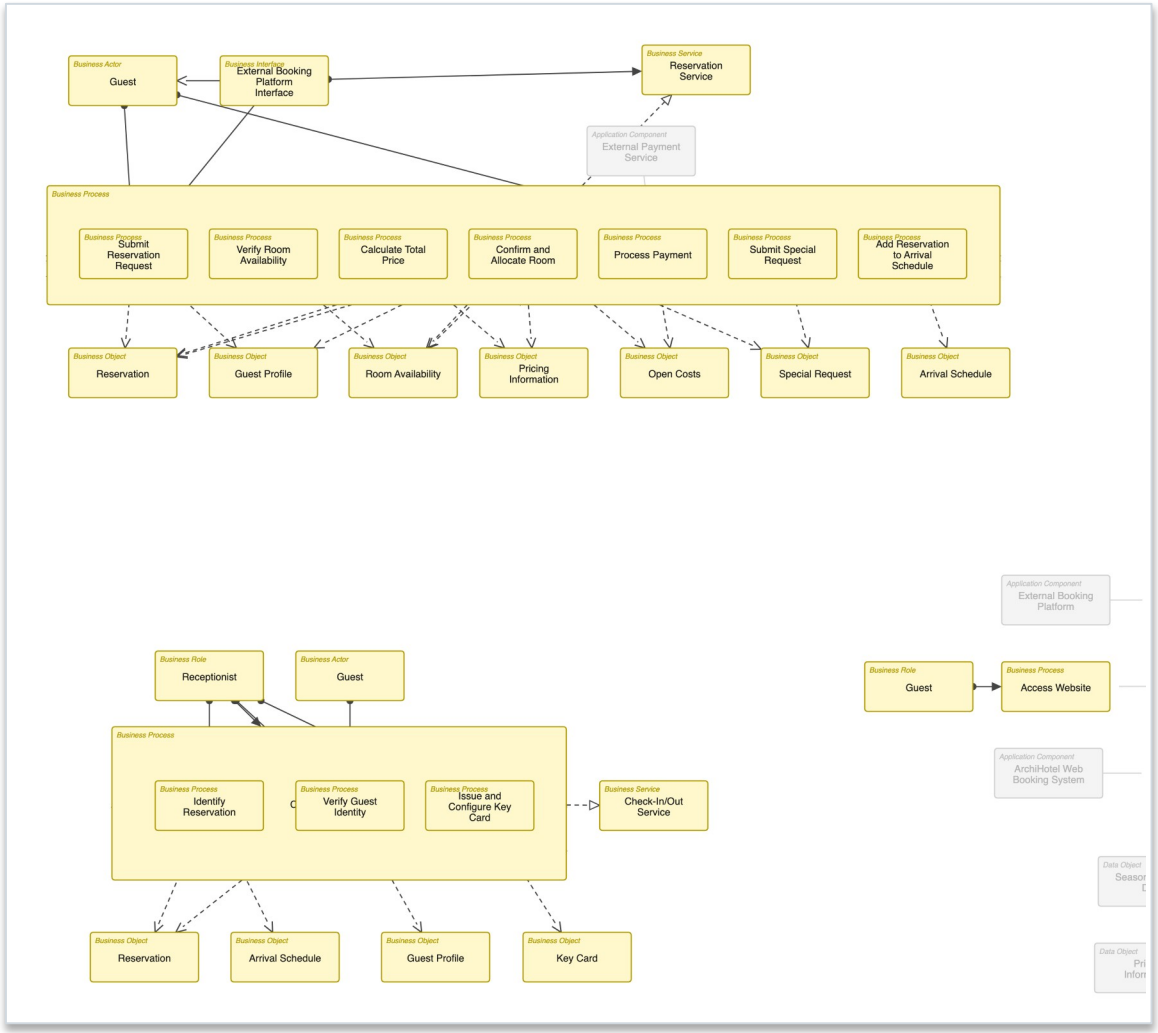
Focus Area F1: From Abstract to Detailed



- F1 = the guest's pre-arrival → arrival journey
- Booking — via the website or an external platform
- Availability & pricing → reserve → external payment
- Special requests → arrival schedule → daily briefing
- Check-in → issue & configure key card
- → Now layer by layer in the detailed model

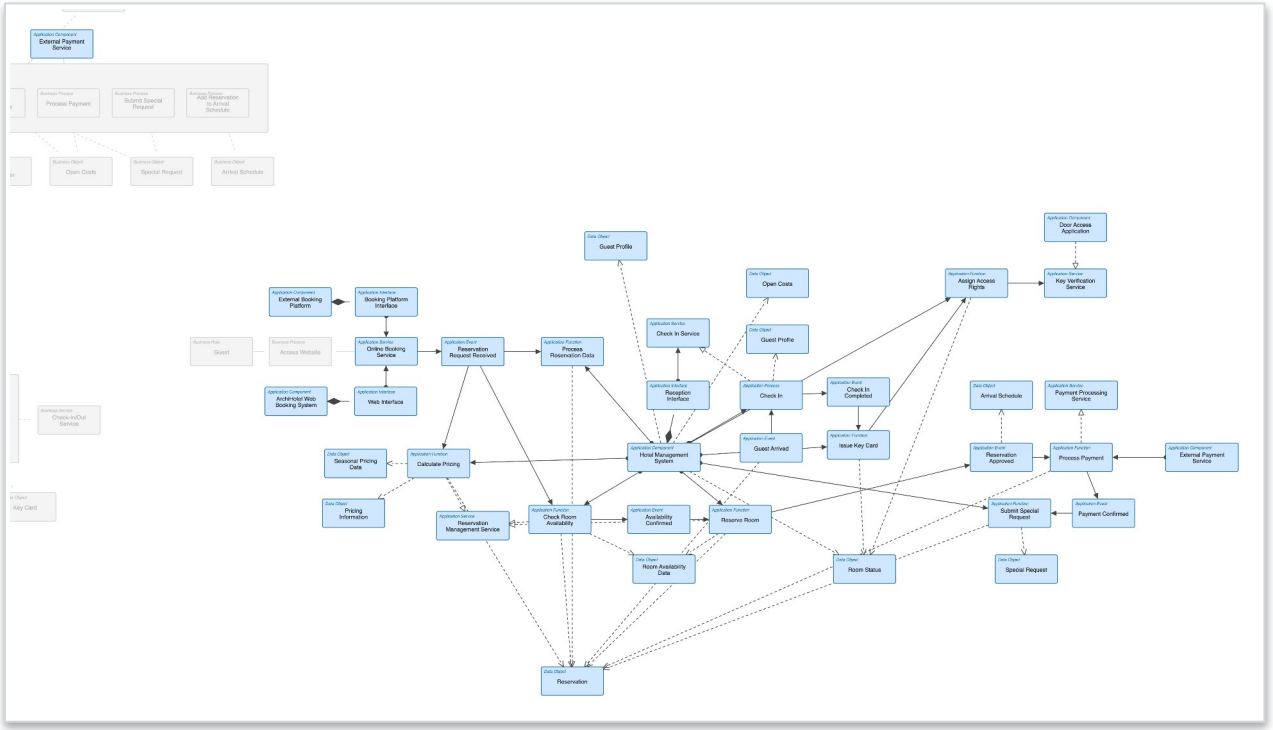
Detailed Model — Business Layer

- Roles: Guest, Receptionist, Hotel Manager, Heads of Concierge & Housekeeping
- Concrete process steps (not one abstract process):
  - submit request → check availability → calculate price → reserve
  - process payment (external) → submit special requests
  - arrival schedule → daily briefing → check-in → issue key card
- Business objects: Reservation, Room Availability, Pricing, Open Costs, Special Request, Arrival Schedule, Briefing Report, Key Card



Detailed Model — Application Layer

- HMS is the central application component
- Internal functions: availability check, pricing, reservation mgmt, payment handling, briefing-report generation, check-in / key-card mgmt
- Around it: Web Booking System, External Booking Platform, External Payment Service, Door Access App
- Data objects: Reservation, Room Availability, Pricing, Guest Profile, Open Costs, Room Status
- Interfaces shown explicitly: Web, Booking-Platform, Payment-Gateway



Detailed Model — Technology Layer

- Nodes: AWS Cloud Server (hosts the core systems);
  - Reception Workstation (PC + phone + payment terminal);
  - Staff Android smartphones; Key-Card Encoder / Reader
- Networks modelled explicitly:
  - Hotel Wi-Fi — reception sync & Staff Portal daily sync
  - Internet — external booking, payment, AWS reach
- Communication paths shown explicitly:
  - Reception ↔ HMS (Wi-Fi); HMS ↔ Payment (Internet);
  - staff phones ↔ HMS (Wi-Fi)

⚠ To be built in Archi

This layer is not yet in the model. Add the Nodes, Networks and communication paths listed at left, then re-run notes/render_archi.py to drop the real diagram here and into the detailed PDF export.

Reflection

Value

- HMS is the visible convergence point
- Change becomes concrete — “what breaks if we swap the payment provider?” = one re-bound link
- Names a shared vocabulary of core objects (Reservation, Open Costs, Arrival Schedule, Briefing Report)

Limitations

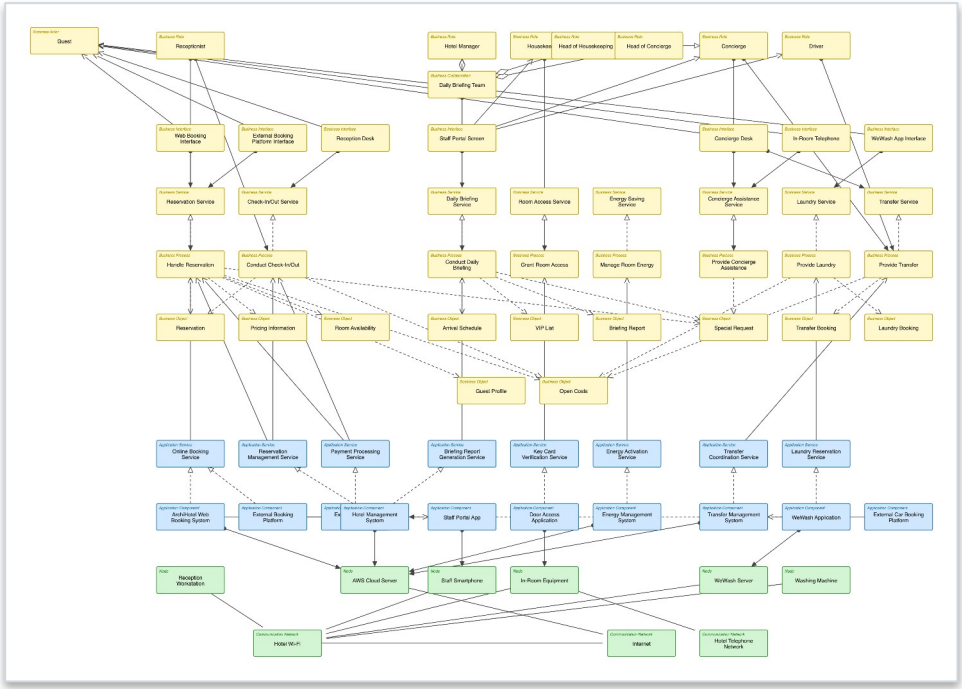
- An as-is snapshot — no governance / update mechanism
- Lossy abstractions (devices collapsed; Association used for Path)
- Material lacks a motivation layer, NFRs/SLAs for the AWS single point of failure, and clear data ownership

Challenges

- Behaviour-element rules (L06) taught after we modelled
- Repeated level-of-abstraction trade-offs
- Archi tooling quirks + a relationship-convention fix (Composition → Assignment)

Summary

- Reservation & Check-In modelled across all three
  - ArchiMate layers, converging on the HMS
- Abstract whole-EA view → detailed F1 view
  - concrete steps · application structure · explicit
  - communication
- Value: shared vocabulary + change visibility
- Limitation: as-is, no motivation layer
- Takeaway: EAM is as much teamwork & abstraction
  - judgment as it is notation



Thank you — questions?